CSCI-UA 381 — Programming Tools for the Data Scientist

Midterm Topics Review

Midterm: Thursday, March 5, 2026 | 60 minutes | 40-50 MCQs | Open Notes (Hardcopy Only)

1. Shell & Command-Line

Core Concepts

The shell is a command-line interface between the user and the operating system. Data scientists prefer it because it is consistent across systems and ideal for automation, especially on remote servers where GUIs are unavailable.

Essential Commands

Command	What It Does
`grep -v 'word' file`	Show lines that do NOT contain 'word'
`grep -w 'word' file`	Match the whole word exactly (not substrings)
`grep -c 'word' file`	Count how many lines contain 'word'
`cut -d',' -f3 file`	Extract the 3rd field using comma as delimiter
`sort file | uniq -c`	Count occurrences of each unique line (must sort first)
`sort file | uniq | wc -l`	Count how many unique lines exist in the file
`sed 's/old/new/g' file`	Replace ALL occurrences of 'old' with 'new' (prints to screen; does not modify file) (use -i to modify file directly)
`sed -n '5,10p' file`	Print only lines 5 through 10
`sed '3,6d' file`	Delete lines 3 through 6
`tr 'a-z' 'A-Z'`	Convert all lowercase letters to uppercase
`awk '{print $1+$2}'`	Print the sum of the 1st and 2nd fields per line `(Assumes whitespace-separated numeric fields.)`
`cmd > file.txt`	Redirect output to file (overwrites existing content)
`cmd >> file.txt`	Append output to file (does not overwrite)

<u>Applying cut and tr Together :</u>

`cut` extracts specific fields from each line of a file. The `-d` flag sets the delimiter and `-f` selects the field number. `tr` translates or replaces characters; commonly used to change letter case. These 2 commands are often chained in pipelines.

<u>Example:</u> extract the 3rd field from a CSV, then convert to uppercase

cut -d',' -f3 data.csv | tr 'a-z' 'A-Z'

If `data.csv` contains a line like `john,25,engineering`, this pipeline extracts `engineering` and outputs `ENGINEERING`.

Key Concept: Why sort before uniq?

uniq only removes adjacent duplicate lines. If the file is not sorted first, non-adjacent duplicates will not be detected. Always pipe through sort first.

```
# Correct way to count unique lines:
sort data.txt | uniq | wc -l

# Correct way to count occurrences:
sort data.txt | uniq -c
```

1. What is the conceptual difference between > and >>?

Ans : **> replaces the entire file.**
>> adds to the end of the file.

2. Why are spaces not allowed in NAME=value?

Ans : In Bash:

NAME=value is assignment.
NAME = value is interpreted as a command called NAME with arguments '=' and 'value'.
Bash is whitespace-sensitive in variable assignment.

3. Why are pipes powerful?

Ans : The pipe | allows:

Small tools to be chained together
Each tool does one job
Output of one becomes input of the next

Pipes & Redirection

The pipe operator | sends the output of one command as the input to the next. This lets you chain multiple tools into a single data processing pipeline.

```
# Example pipeline: find most frequent ERROR IP
grep 'ERROR' logs.txt | cut -d' ' -f3 | sort | uniq -c | sort -rn | head -n 1
```

Shell Variables & Control Flow

Shell scripts are plain text files containing a sequence of commands. They let you automate repetitive tasks. Every shell script starts with a shebang line that tells the OS which interpreter to use.

Variables

Assign a value with `NAME=value` (no spaces around =). Read a variable with `$NAME` or `${NAME}`. Common pitfall: spaces around = cause a syntax error.

```
#!/bin/bash
# Assign a variable (NO spaces around =)
name="Alice"
age=25
# Read a variable using $
echo "Hello, $name. You are $age years old."
# Output: Hello, Alice. You are 25 years old.
# WRONG (causes error): name = "Alice"  -- spaces not allowed
```

if / else Statement

Bash conditions use `[ condition ]` with spaces inside the brackets. Use `-eq` for numbers, = for strings. Always close with `fi`.

```
score=85
if [ $score -ge 90 ]; then
    echo "Grade: A"
elif [ $score -ge 80 ]; then
    echo "Grade: B"
else
    echo "Grade: C or below"
fi
# Output: Grade: B
```

Numeric comparison operators: `-eq` (equal), `-ne` (not equal), `-gt` (greater than), `-lt` (less than), `-ge` (greater or equal), `-le` (less or equal).

2. Python Fundamentals

Data Structures (IMPORTANT)

Structure	Key Properties
`List []`	Ordered, mutable, allows duplicates, supports indexing & slicing
`Tuple ()`	Ordered, immutable (cannot be changed), supports indexing & slicing
`Set {}`	Unordered, mutable, NO duplicates, NO indexing
`Dict {k:v}`	Key-value pairs, insertion-ordered (Python 3.7+), mutable, keys must be immutable

Slicing Examples

Slicing syntax: list[start : stop : step]

Slice	What It Returns
`lst[1:4]`	Elements at indices 1, 2, 3 (stop is exclusive)
`lst[::2]`	Every other element starting from index 0

`lst[::-1]`	The entire list reversed
`lst[1:4:2]`	Elements at indices 1 and 3 (step of 2)

Chaining Indexing with Slicing (2D Lists) : You can index into a nested list first, then slice the result in the same line:

Example:

```
scores = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]


print(scores[0][::-1])  # Output: [30, 20, 10]
print(scores[-1][::2])  # Output: [70, 90]
print(scores[1][1:])    # Output: [50, 60]
```

Code breakdown :

scores[0][::-1] — scores[0] grabs the first sublist [10, 20, 30]. Then [::-1] reverses it
→ [30, 20, 10].

scores[-1][::2] — scores[-1] grabs the last sublist [70, 80, 90]. Then [::2] takes every other element starting from index 0
→ [70, 90].

scores[1][1:] — scores[1] grabs the middle sublist [40, 50, 60]. Then [1:] slices from index 1 to the end
→ [50, 60].

The mental model: Always read left to right. The first [...] picks which row. The second [...] operates on that row exactly like a regular list slice.

Set Operations

Operation	Meaning
`s1 | s2`	**Union** — all elements from both sets
`s1 & s2`	**Intersection** — only elements in both sets
`s1 - s2`	**Difference** — elements in s1 but NOT in s2
`s1 ^ s2`	**Symmetric difference** — elements in either set but NOT both

The `|` operator and `.union()` method are equivalent. The same applies to all set operations:

Example:

```
s1 = {'apple', 'banana', 'cherry'}
s2 = {'mango', 'banana', 'grape'}
print(s1.union(s2))                    # {'apple', 'banana', 'cherry', 'mango', 'grape'}
```

```
print(s1.intersection(s2))        # {'banana'}
print(s1.difference(s2))          # {'apple', 'cherry'}
```

Note: Since sets have NO duplicates, even if the same element appears in both sets, it will only appear **once** in the union result.

Roles in a Data Science Team:

Data science work is typically divided across 3 specialist roles, each responsible for a distinct stage of the data pipeline:

Role	Primary Responsibility
Data Engineer	Builds and maintains pipelines that collect, store, and move data
Data Analyst	Explores data and produces insights and reports to support decisions
ML Engineer	Builds, trains, and deploys predictive models

Data Types and File Formats

Data comes in 3 structural categories:

Structured data has a strict, predefined schema; rows and columns with consistent types. Examples: SQL tables, Excel spreadsheets, CSV files

Semi-structured data has some organization but doesn't conform to a rigid schema. It uses self-describing tags or key-value pairs, allowing flexible nesting. Example: JSON (JavaScript Object Notation).

Unstructured data has no predictable format or pattern. Examples: images, raw text files, audio.

Lambda & List Comprehensions

Lambda is a concise, anonymous, single-expression function. Use it for short throwaway operations, especially with map() and filter(). Lambda functions are expressions, not statements.

A list comprehension is a concise way to build a new list by applying an expression to each item in an iterable. It replaces the common pattern of creating an empty list, looping, and appending — all in one readable line.

Why list comprehensions? :

1. They are fast: List comprehensions use localized variables internally and avoid the calling of .append() on every iteration. For large datasets, this difference is measurable.

2. They are expressions, not statements: This means a list comprehension produces a value directly. You can assign it, pass it as an argument, or nest it; just like any other expression. A traditional for loop cannot do this.

Examples:

```
# Regular function vs lambda
def square(x):
    return x ** 2
print(square(5))

Lambda:
square = lambda x: x ** 2
print(square(5))  # output = 25


# map() example — apply function to every element
nums = [1, 2, 3, 4]

squared = list(map(lambda x: x**2, nums))
print(squared)  # Output = [1, 4, 9, 16]


# filter() example — keep elements that satisfy condition
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)  # Output = [2, 4]
```

- **List comprehension syntax :**

*new_list = [expression **for** item in iterable]*

This is equivalent to :

new_list = []
for item in iterable:
  new_list.append(expression)

- **With a Condition (Filtering) :**

You can filter elements using an IF at the end: ***[expression for item in iterable if condition]***

Example :

```
print([x*2 for x in range(5) if x % 2 == 0])   # Output : [0, 4, 8]
```

Equivalent for-loop:

```
result = []
for x in range(5):
    if x % 2 == 0:
        result.append(x * 2)
print(result)
```

3. NumPy Conceptual Understanding & Operations

Why are NumPy arrays faster than Python lists?
Ans: Because:

- They store data in contiguous memory
- They support vectorized operations
- They are implemented in C under the hood

Lists:

- Store references to objects
- Allow mixed types
- Are slower for numerical computation

What does "homogeneous dtype" mean?
Ans: All elements in a NumPy array must have the same data type.
If mixed types are given: *np.array([1, 2, 3.5]) ;* NumPy automatically upcasts everything to float.

Numpy Broadcasting:

Broadcasting allows NumPy to perform element-wise operations on entire arrays without writing an explicit loop. The operation is applied to every element automatically.

Example:

```
import numpy as np

temps = np.array([22, 25, 19, 30])
adjusted = temps + 5
print(adjusted)                    # Output : [27, 30, 24, 35]
```

Without broadcasting, you would need a for loop to add 5 to each element. NumPy does this internally in C, which is why it is significantly faster than a Python loop.

NumPy provides the `ndarray` — a fast, efficient, homogeneous array for numerical data. A NumPy array has a single data type (`dtype`), and values are automatically upcast if mixed types are provided (e.g., integers and floats become floats).

Operation	What It Does
`np.array([1,2,3])`	Create a 1D array
`np.array([[1,2],[3,4]])`	Create a 2D array
`arr.shape`	Returns a tuple representing the array's dimensions. For example: (n,) for a 1D array of length n, or (rows, columns) for a 2D array
`arr[arr > 3]`	Boolean filtering; returns elements where condition is True
`arr[1:4]`	Slice; elements at indices 1, 2, 3
`arr.sum() / arr.mean()`	Sum or mean of all elements
`np.dot(a, b)`	Matrix multiplication (dot product)

Boolean filtering Example:

```
import numpy as np
arr = np.array([1,2,3,4,5])
print(arr[arr % 2 == 0])
```

Output : [2 4]

Explanation: `arr % 2 == 0` creates a boolean mask. NumPy returns the **values where the condition is True**.

4. Pandas

Core Concepts

What is the relationship between DataFrame and Series?

- A DataFrame is a collection of Series.
- Each column is a Series.
- All columns share the same index.

Think of:

- DataFrame → table
- Series → single column

Why does df['col'] return a Series but df[['col']] return a DataFrame?

Because:

- Single brackets → select one column → Series
- Double brackets → select multiple columns (list) → DataFrame

Conceptual difference: merge vs concat

concat:

- Stacks DataFrames
- Does not require a key
- Combines based on axis

merge:

- Joins based on a shared column
- Similar to SQL join

A DataFrame is a 2D table where each column is a Series. Every column shares the same index. The index is the 'address' of each row.

Method	What It Does
`df.shape`	Returns (rows, columns)
`df.info()`	Shows column types, non-null counts, memory usage
`df.describe()`	Summary statistics for numerical columns
`df.head() / df.tail()`	First or last 5 rows
`df['col']`	Select a single column (returns Series)
`df[['col1','col2']]`	Select multiple columns by passing a list (returns DataFrame)
`df[df['col'] > 5]`	Filter rows where condition is True

`df.loc[row_label, col_label]`	Select by row and/or column label name
`df.iloc[0]`	Select by integer position

Handling Missing Values (NaN)

Method	Behavior
`df.isna().sum()`	Count missing values per column
`df.dropna(how='any')`	Drop row if ANY value is missing (aggressive)
`df.dropna(how='all')`	Drop row only if ALL values are missing (conservative)
`df['col'].interpolate(method='linear')`	Fill NaN by estimating from neighboring value
`df.fillna(0)`	Replace all NaN with 0
`df['col'].fillna(df['col'].mean())`	Replace NaN with the column mean
`df['col'].replace(0, np.nan)`	Standardize sentinel values (like 0 or -1) into proper NaN before cleaning

When to use which:

- Use **fillna()** when you want every NaN replaced with the same fixed value or statistic.
- Use **replace()** when your data uses placeholder values like 0, -1, or "" to represent missing data — these won't be caught by dropna() or fillna() until you convert them to actual NaN first.
- Use **interpolate()** when the column has a natural progression and the missing value should logically fall between its neighbors, rather than being replaced by a fixed constant.

Setting and Resetting Index

set_index() promotes a column to become the row index. This is useful for fast label-based lookups with loc. But, it permanently discards the previous default integer index. It cannot be recovered unless you saved it or call **reset_index()** before setting a new one.

Example:

df = df.set_index('employee_id') # employee_id is now the index

The old 0, 1, 2... integer index is gone

df = df.reset_index() # brings employee_id back as a column, restores integer index

GroupBy — Split, Apply, Combine

groupby splits the DataFrame into groups, applies a function to each group, then combines the results back into a new DataFrame. By default, results are sorted alphabetically by the group key (sort=True).

```
df.groupby('dept')['salary'].mean()  # mean salary per dept
df.groupby('dept')['salary'].sum()   # total salary per dept
df.groupby('dept').size()            # count rows per dept
```

Multi-Column GroupBy :

When grouping by multiple columns, the result has a multi-level index. The first column in the list becomes the outer (primary) index, and the second becomes the inner index. The visual hierarchy in the output directly reflects the order you pass the columns.

Example :

df.groupby(['country', 'city'])['population'].mean()

country is the outer index (primary level)

city is the inner index (secondary level)

Order in the list controls which becomes primary

Breakdown:

- df.groupby(['country', 'city']) — split the DataFrame into groups. Every unique combination of country + city becomes its own group. country is listed first so it becomes the outer (primary) index in the result.
- ['population'] — from each group, select only the population column.
- .mean() — calculate the average population for each country-city combination.

So the result looks like:

```
country   city
Brazil    Sao Paulo      12000000
          Rio             6700000
USA       New York        8300000
          Los Angeles     4000000
```

Combining DataFrames

Method	What It Does
`pd.concat([df1,df2], axis=0)`	Stack rows vertically (same columns)
`pd.concat([df1,df2], axis=1)`	Combine columns side by side (same rows)
`pd.merge(df1,df2,on='id',how='inner')`	Keep only rows with matching key in both
`pd.merge(df1,df2,on='id',how='left')`	Keep all rows from df1, match where possible
`pd.merge(df1,df2,on='id',how='outer')`	Keep all rows from both, fill gaps with NaN

5. Data Visualization

Choosing the Right Chart

How to choose the correct chart?
Ask:

1. Is the variable categorical or numerical?
2. Are we showing distribution or relationship?
3. Are we comparing groups?

When to use each chart conceptually :

Histogram:

- Use for continuous numerical data
- Shows distribution shape (normal, skewed, etc.)

Bar chart:

- Use for categorical comparisons
- Each bar represents a category

Scatter plot:

- Use when studying relationship between two numeric variables
- Detect correlation or patterns

Line plot:

- Use for time series or continuous trends

Boxplot:

- Shows spread, median, outliers
- Useful for comparing distributions across categories

Pairplot:

- Use when you have multiple numerical variables and want to see all pairwise relationships at once in a single view

Regplot / Implot:

- Use when you want to show the relationship between two numeric variables with a trend line overlay
- Key distinction from scatter plot: Regplot shows direction and fit, not just the raw points

Heatmap:

- Use to show correlation or intensity across a matrix
- Useful when you have many variables and want to spot patterns across all of them at once

FacetGrid:

- Use when you want to show the same type of plot repeated across different subsets of your data
- Example: the same histogram plotted separately for each department

Chart Type	When to Use It
`plt.hist()`	Show the distribution/frequency of a single numerical variable
`plt.bar() / plt.barh()`	Compare values across categories
`plt.scatter()`	Show the relationship between two numerical variables
`plt.pie()`	Show proportions of a whole (parts add up to 100%)
`plt.boxplot()`	Show median, quartiles, and outliers of a dataset
`plt.plot()`	Show trends over time or continuous data
`plt.violinplot()`	Show distribution density (like boxplot + histogram combined)

Advanced Chart Types for Exploration:

CHART	When to Use
PAIRPLOT	Quickly visualize relationships between all pairs of numerical variables in a dataset at once
HEATMAP	Show correlation or intensity across a matrix — useful for spotting patterns across many variables
FACETGRID	Create a grid of subplots, each showing the same type of plot for a different subset of the data
REGPLOT / LMPLOT	Scatter plot with an overlaid trend line — use when you want to visualize the relationship between two variables and see the direction and fit of that relationship

---

Super Quick Reference — Must-Know Concepts

- **List vs Tuple:** Lists are mutable (changeable), tuples are immutable (fixed)
- **Set vs List:** Sets have no duplicates and no guaranteed order; lists allow both
- **sort before uniq:** uniq only detects adjacent duplicates, so input must be sorted first
- **> vs >>:** > overwrites a file, >> appends to it
- **axis=0 vs axis=1 in concat:** axis=0 adds rows (vertical), axis=1 adds columns (horizontal)
- **merge vs concat:** merge joins on a shared key column; concat stacks without a key
- **dropna how='any' vs 'all':** 'any' drops if one value is missing; 'all' drops only if all are missing
- **loc vs iloc:** loc uses label names; iloc uses integer positions.
- **lambda vs def:** lambda is a single-expression anonymous function for short operations.
- **groupby output order:** By default, groupby sorts group keys alphabetically (sort=True). This can be disabled using sort=False.
- **Shell variables:** Assign with NAME=value — no spaces around =. Read with $NAME. Spaces around = cause a syntax error.

---

*All the best on Thursday! Reach out to me **anytime** at rss9385@nyu.edu with questions.*

Rahil Shaikh — TA, CSCI-UA 381